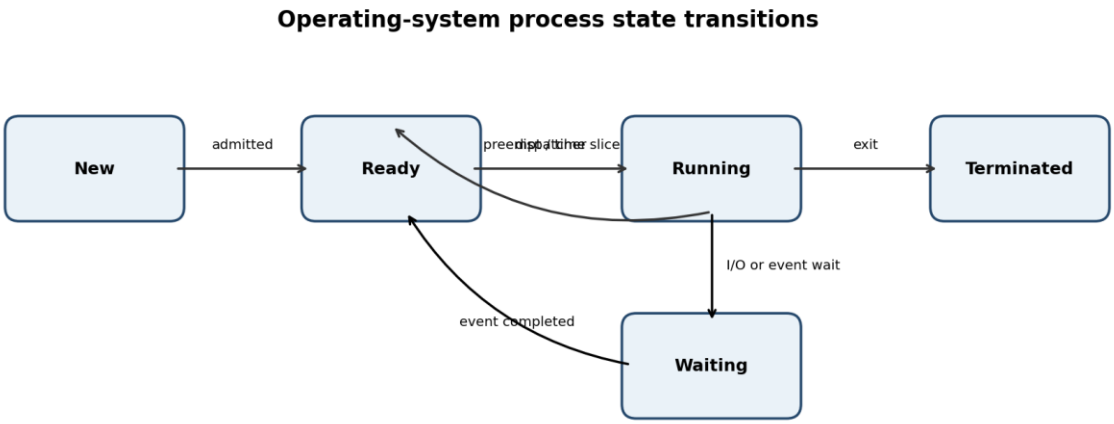
OPERATING SYSTEMS

ASSIGNMENT 2 – CO2

Process Management, Scheduling, Threads, IPC, Synchronisation and Deadlocks



Submitted in partial fulfilment of the Operating Systems course assignment

Student details	Information
Name	S. Maruthu Pandi
Register Number	192511162
Course	CSA0421 – Operating Systems
Assignment	Assignment 2 – CO2
GitHub Repository	https://github.com/Maruthupandi-maker/CSA0421-Operating-System-tree/main/CSA0421%20192511162%20Assignment%202

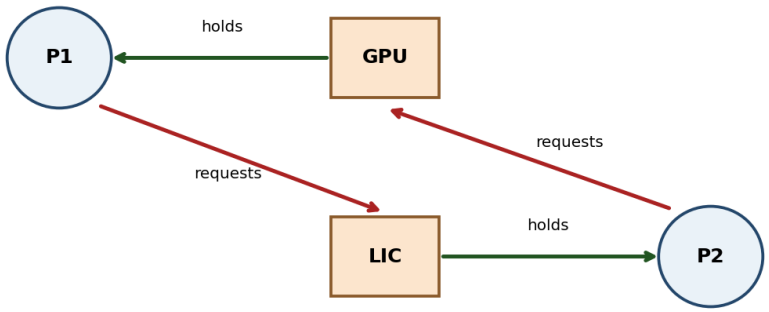
Q6 – DEADLOCK HANDLING AND BANKER'S ALGORITHM

6.1 Deadlock definition and four necessary conditions

A deadlock is a situation in which a set of processes are permanently blocked because each process is waiting for a resource held by another process in the same set. In the laboratory cluster, one job may hold a GPU while waiting for a software licence, while another holds the licence while waiting for a GPU.

Necessary condition	Meaning	Laboratory example
Mutual exclusion	At least one resource is non-shareable and can be used by only one process at a time.	A particular GPU allocation or floating licence token can be assigned to one job at a time.
Hold and wait	A process holds one or more resources while requesting additional resources.	A simulation holds a GPU while waiting for a licence.
No pre-emption	A resource cannot be forcibly taken away; it is released voluntarily.	The scheduler cannot safely reclaim a running GPU allocation without stopping or checkpointing the job.
Circular wait	A circular chain exists where each process waits for a resource held by the next process.	Job P1 waits for a licence held by P2, while P2 waits for a GPU held by P1.

Resource-allocation cycle illustrating deadlock



P1 holds GPU and waits for licence; P2 holds licence and waits for GPU.

Figure 6. Resource-allocation cycle: resource-to-process arrows are allocations; process-to-resource arrows are requests.

For a resource-allocation graph, circles represent processes and squares represent resource types. An allocation edge points from a resource instance to the process holding it. A request edge points from the process to the resource it needs. With one instance of each resource type, a cycle is sufficient to establish deadlock. With multiple instances, a cycle indicates possible deadlock but is not by itself sufficient to prove that all processes are deadlocked.

6.2 Completed Banker's safety algorithm

Banker's algorithm is a deadlock-avoidance algorithm. It grants a request only if the resulting state remains safe. A safe state has at least one sequence in which every process can obtain its remaining resources, finish and release its allocated resources.

The assignment supplies five processes and three resource types. Available = (3,3,2). Need is computed as Need = Maximum − Allocation. The safety check repeatedly searches for an unfinished process whose Need is less than or equal to Work for every resource type. When found, its Allocation is returned to Work and it is marked finished.

```
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3
int avail[R] = {3, 3, 2};
int maxm[P][R] = {
    {7, 5, 3}, {3, 2, 2}, {9, 0, 2},
    {2, 2, 2}, {4, 3, 3}
};
int alloc[P][R] = {
    {0, 1, 0}, {2, 0, 0}, {3, 0, 2},
    {2, 1, 1}, {0, 0, 2}
};

int is_safe(int seq[P]) {
    int work[R], finish[P] = {0}, count = 0;
    memcpy(work, avail, sizeof work);
    while (count < P) {
        int found = 0;
        for (int p = 0; p < P; ++p) {
            if (finish[p]) continue;
            int possible = 1;
            for (int r = 0; r < R; ++r) {
                int need = maxm[p][r] - alloc[p][r];
                if (need > work[r]) { possible = 0; break; }
            }
            if (possible) {
                for (int r = 0; r < R; ++r) work[r] += alloc[p][r];
                finish[p] = 1;
                seq[count++] = p;
                found = 1;
            }
```

```
        }
        if (!found) return 0;
    }
    return 1;
}

void request(int p, const int req[R]) {
    for (int r = 0; r < R; ++r) {
        if (req[r] > maxm[p][r] - alloc[p][r] || req[r] > avail[r]) {
            printf("P%d request: must wait (not available)\n", p);
            return;
        }
    }
    for (int r = 0; r < R; ++r) {
        avail[r] -= req[r]; alloc[p][r] += req[r];
    }
    int seq[P];
    if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence", p);
        for (int i = 0; i < P; ++i) printf(" P%d", seq[i]);
        printf("\n");
    } else {
        for (int r = 0; r < R; ++r) {
            avail[r] += req[r]; alloc[p][r] -= req[r];
        }
        printf("P%d request: denied (unsafe)\n", p);
    }
}

int main(void) {
    int seq[P];
    if (is_safe(seq)) {
        printf("initial state safe:");
```

```
        for (int i = 0; i < P; ++i) printf(" P%d", seq[i]);
        printf("\n");
    } else printf("initial state unsafe\n");
    request(1, (int[R]){1, 0, 2});
    request(4, (int[R]){3, 3, 0});
    request(0, (int[R]){0, 2, 0});
    return 0;
}
```

Compile and execute:

```
gcc -std=c11 -Wall -Wextra banker.c -o banker
./banker
```

Expected output:

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

6.3 Verify the supplied state by hand

Initially Work = Available = (3,3,2). For the standard state, process P1 (index 1) needs (1,2,2) and can finish; its allocation (2,0,0) is returned, giving Work = (5,3,2). P3 then needs (0,1,1) and releases (2,1,1), giving (7,4,3). P4 needs (4,3,1) and releases (0,0,2), giving (7,4,5). P0 then needs (7,4,3) and releases (0,1,0), giving (7,5,5). Finally P2 can finish. Thus one safe sequence is P1, P3, P4, P0, P2.

6.4 Analyse each request

Request	Check	Decision
P1 requests (1,0,2)	The request is no greater than P1's remaining Need and no greater than Available. Tentatively allocate it and run the safety algorithm.	Grant. The resulting state remains safe; sequence P1, P3, P4, P0, P2.

P4 requests (3,3,0)	After the previous grant, Available is (2,3,0). P4 asks for 3 units of resource A, but only 2 are available.	Must wait. The request exceeds currently available resource A.
P0 requests (0,2,0)	Available is (2,3,0), so the request is currently available and is within P0's remaining claim. Tentatively grant it, leaving Work=(2,1,0).	Deny. The resulting state is unsafe because no unfinished process can satisfy its remaining Need with Work=(2,1,0). Roll back the tentative allocation.

For P0's request, after tentative allocation P0's Allocation becomes (0,3,0) and its Need becomes (7,2,3); it cannot finish with Work=(2,1,0). P1 needs (0,2,0), P3 needs (0,1,1), P2 needs (6,0,0), and P4 needs (4,3,1). Each requires at least one resource amount greater than Work. Therefore no process can be selected first, the safety scan fails, and the request must be rolled back. An unsafe state does not always mean that deadlock has already occurred; it means the OS cannot guarantee completion for all processes.

6.5 Compare deadlock strategies

Strategy	**Method**	**Strength**	**Weakness / suitability**
Prevention	Break at least one of the four necessary conditions. For example, impose a global resource acquisition order to break circular wait.	Can make deadlock impossible under the chosen rules.	May reduce utilisation or require jobs to request all resources up front; strict ordering can be inconvenient.
Avoidance	Use Banker's algorithm and grant only requests that preserve a safe state.	Prevents entry into states that cannot be proven safe.	Requires maximum demands in advance and incurs a safety check for resource requests.

Detection and recovery	Allow allocations, periodically detect cycles/insufficient progress, then recover by aborting or rolling back selected jobs.	Flexible when maximum demands are unknown; resources can be used more freely before deadlock.	Detection costs CPU time; aborting a long simulation loses work unless checkpoints exist.
Ignore (ostrich approach)	Do not perform special deadlock handling; rely on rare occurrence or administrator intervention.	No runtime prevention/avoidance overhead.	Unacceptable for critical cluster jobs if a deadlock can stop important workloads.

6.6 Complexity prediction

Let n be the number of processes and m be the number of resource types. A safety pass may scan up to n processes, and for each process compare up to m resource values. In the worst case, the algorithm may perform up to n successful passes, so the conservative time complexity is $O(m \times n^2)$. The matrices for Maximum, Allocation and Need require $O(nm)$ storage; the Work vector requires $O(m)$, and Finish/Sequence require $O(n)$.

For a single resource request, checking the request bounds costs $O(m)$. If the request is tentatively allocated, the safety test dominates, so the overall worst-case decision is $O(mn^2)$.

6.7 Measurement plan for 10, 100 and 1,000 processes

To measure, generate valid Maximum and Allocation matrices for each process count, initialise Available, call the safety algorithm repeatedly, and time it with clock_gettime(CLOCK_MONOTONIC). Use the same resource count m for all three tests and repeat enough times to reduce timing noise. Record the compiler flags and processor details.

6.8 Alternative: detection and recovery instead of Banker’s algorithm

With detection and recovery, the cluster does not reject every allocation that might lead to an unsafe state. Instead, the OS or resource manager periodically runs a detection algorithm. When it finds a deadlock, it chooses one or more victim jobs to abort, restart or roll back to a checkpoint.

- Banker’s avoidance performs a safety check when a request is considered; the brief gives worst-case $O(m \times n^2)$ for a safety check.
- Detection has a recurring cost, often based on a resource-allocation or wait-for graph. If it runs every few minutes, the check is not necessarily paid for every request, but deadlock may continue until the next scan.
- Recovery can waste completed computation. Checkpointing reduces lost work but requires storage and restart management.

For the research-centre cluster, a hybrid is appropriate: use resource ordering and reservations for predictable high-value resources, apply Banker-style avoidance to jobs with reliable declared maximum claims, and use detection/recovery with checkpoints for flexible jobs whose exact future demand is unknown.

6.9 Final defence

Banker’s algorithm is suitable when the number of processes and resource types is moderate and jobs can declare maximum resource needs in advance. It gives a formal safe-state guarantee, but its $O(mn^2)$ worst-case checking cost and need for maximum-demand information can limit scalability. For the shared laboratory cluster, a hybrid policy gives better practicality: prevent obvious circular waits with a fixed resource order, reserve GPUs/licences for known jobs, use avoidance for jobs with predictable claims, and enable detection plus checkpoint-based recovery for the remaining workload. This balances safety, utilisation and recoverability.